LatchMoE: Graph-Compatible Expert Offloading through Address-Stable Residency

Anonymous Author(s)

Abstract

Sparse mixture-of-experts models bound per-token arithmetic but retain a large parameter footprint. Expert offloading addresses this footprint, yet a bounded expert cache changes weight placement as routing changes, conflicting with the stable storage bindings expected by graph replay. Our key insight is that replay requires stable graph-visible storage, not static logical expert residency. We present LatchMoE, a capacity-adaptive piecewise runtime: capacity-fitting routed work replays captured expert computation from fixed persistent slots, whereas oversized prefills execute exact eager waves from fixed transient banks. The overflow path is selected deliberately rather than entered after failed replay. Both paths use versioned leases to order transfer readiness, binding publication, computation, and reuse. A vLLM-Ascend implementation evaluates the three target models under 11 semantic, graph, and lifecycle gates; Qwen3-Next's native full-resident end-to-end subcheck is capacity-limited on this device. On Qwen3-30B-A3B, bounded waves change paired TTFT by -35.13% in an internal comparison against graph-compatible full-layer staging; its 95% CI is [-35.67%, -34.60%]. Enabling overlap in a one-factor campaign changes TTFT by -27.87%; its 95% CI is [-30.18%, -25.24%]. Performance results are scoped to one Ascend 910B2, BF16, tensor parallelism one, and serving concurrency one. Against full residency, the same workload trades a median sampled HBM peak from 97% to 80% for +261.28% TTFT and +309.51% TPOT; the reported TTFT reductions therefore characterize capacity-bounded staging rather than a universal end-to-end speedup.

1 Introduction

Mixture-of-Experts (MoE) architectures have emerged as a prominent approach for scaling Large Language Models (LLMs) to hundreds of billions of parameters [2, 5, 8, 16]. By dynamically routing each token to only a sparse subset of experts, MoE models substantially increase model capacity while keeping per-token computation bounded [3, 15]. However, sparse activation reduces computation rather than model storage: although only a few experts are executed for a given token, the weights of all experts must remain accessible because any expert may be selected at runtime. As MoE models scale, the complete expert pool can exceed accelerator High-Bandwidth Memory (HBM) capacity. To serve such models on memory-constrained accelerators, **expert offloading** has become an important strategy: the runtime keeps a portion of expert weights in host memory and accesses or stages them on demand during inference [7, 10, 25].

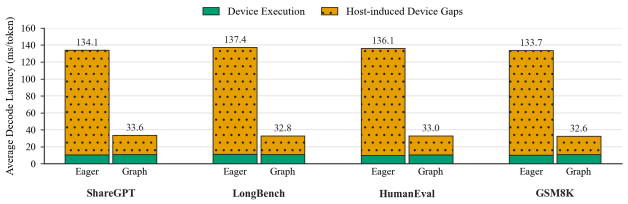

Figure 1. Average decode latency breakdown under eager execution and graph replay. Graph replay substantially reduces host-induced device gaps while leaving device execution time nearly unchanged.

To make expert offloading practical, prior systems reduce its data-movement overhead through expert caching, routing-aware prediction and prefetching, and overlap between transfers and computation [1, 4, 7, 9, 25]. These techniques reduce the frequency of host-to-device transfers or hide part of their latency. However, they do not remove the execution dynamism of cache-based expert offloading: expert admission and eviction in a bounded HBM cache can change device storage assignments across inference steps.

Such runtime changes can be naturally accommodated under eager execution, where the host resolves the current device storage assignments and dispatches the corresponding kernels at each decoding step. This flexibility, however, comes at a substantial performance cost. Autoregressive decoding repeatedly executes a largely recurring sequence of fine-grained accelerator kernels, and launching these kernels individually incurs repeated host-side scheduling, dispatch, and synchronization overhead. **Graph replay** mitigates this overhead by capturing the recurring execution sequence once and replaying it as a unit, thereby reducing host-induced gaps between kernel executions [17, 26].

As shown in Figure 1, graph replay reduces average decode latency by approximately 75% across four workloads, from 133.7–137.4 ms/token under eager execution to 32.6–33.6 ms/token, while device execution time remains nearly unchanged. This result indicates that most of the improvement comes from eliminating host-induced device idle time rather than accelerating the underlying kernels.

However, graph replay requires the storage bindings referenced by captured computation to remain replay-stable.

A dynamically managed expert cache can violate this requirement when routing-driven expert admission and eviction reassign logical experts to different device storage locations that are directly referenced by captured kernels. If such reassignment changes the storage bindings observed by the captured computation, the runtime must either repair or recapture the graph to reflect the new bindings, or bypass replay and fall back to eager execution. These options reintroduce host-side intervention and diminish the benefit of graph replay. The resulting systems problem is to **support routing-driven expert replacement within a bounded HBM cache while keeping the storage bindings observed by replayed expert computation stable**.

Existing graph-compatible offloading mechanisms solve only parts of this problem. On CUDA platforms, UVA provides stable access to host-resident weights, but does not support selective expert replacement within a bounded HBM cache [13, 22]. Prefetch-based offloading can preserve replay with preallocated device buffers [21]. Applied to MoE, however, layer-wise staging must move the entire expert layer despite sparse expert activation, incurring unnecessary HBM capacity and transfer costs. Exploiting this sparsity requires expert-granular staging, where the required experts become routing-dependent and may exceed the available device slots. Existing approaches therefore do not directly support routing-driven expert replacement over bounded, replay-stable HBM slots.

Our key insight is that **runtime dynamism need not change the execution state captured by the graph**. Graph replay requires stable storage bindings, not static expert residency. We therefore decouple logical expert identity from device storage. For each MoE layer whose experts are offloaded, fixed HBM slots remain bound to captured computation while routing-dependent experts dynamically occupy and leave them. This abstraction allows expert residency to evolve without changing the storage interface used by graph replay. When routed demand exceeds that replay capacity, LatchMoE preserves exact semantics through a separate bounded overflow path rather than rebinding the captured graph.

Realizing this abstraction raises three systems challenges. *Challenge 1*: **Routing-dependent expert placement must not change the storage bindings used by captured computation.** The experts required by a layer are revealed only after routing, yet routing-dependent residency decisions must not change the storage bindings or execution structure observed by captured expert computation. *Challenge 2*: **The routed expert working set can exceed the available execution-bank capacity.** Staging all required experts at once violates the memory budget, whereas fetching them sequentially exposes repeated weight-transfer latency on the inference critical path. *Challenge 3*: **Slot reuse must remain correct under asynchronous transfer and computation.** Reassigning a slot before its previous consumer finishes can overwrite weights still in use, while exposing a new assignment before transfer completion can make computation observe incomplete contents.

We present LatchMoE, a capacity-adaptive piecewise MoE offloading runtime. It routes once and then selects one of two expert paths. Capacity-fitting work uses replay-stable persistent slots, allowing captured expert computation to retain its storage bindings while logical owners change. An oversized prefill instead executes exact eager waves from fixed transient banks and reassembles their outputs before native composition. This planned overflow mode is not a fallback from failed graph replay. LatchMoE pipelines staging of subsequent waves with ongoing computation. Finally, versioned slot leases coordinate transfer readiness, computation completion, and ownership reassignment, preventing a slot from being reused before its previous lifetime ends or exposed before its new contents are ready.

In summary, this paper makes the following contributions:

- We identify that the conflict between dynamic expert offloading and graph replay arises when changing expert placement also changes the storage bindings referenced by captured computation. We show that graph replay requires stable storage bindings, not static expert residency, enabling dynamic experts to be virtualized over a fixed storage interface.
- We design LatchMoE, a capacity-adaptive piecewise MoE offloading runtime. Its replay path virtualizes routing-dependent logical ownership over persistent, replay-stable HBM slots; its overflow path executes oversized prefills as exact eager waves from bounded transient banks. Versioned slot leases coordinate asynchronous transfer, computation, and storage reuse on both paths.
- We implement LatchMoE on vLLM-Ascend and validate its execution semantics across three MoE architectures. The evaluated units preserve exact outputs and their configured piecewise contract: graph-designated pieces replay as required, while overflow expert waves use the explicit eager path without unintended fallback or lifecycle violations. Performance claims remain scoped to a matched single-NPU Qwen3-30B-A3B configuration, where multi-wave execution reduces prefill TTFT relative to graph-compatible full-layer staging.

2 Background and Problem Formulation

2.1 MoE Inference and Cache-Based Expert Offloading

A Mixture-of-Experts (MoE) layer replaces the dense feed-forward network with a set of experts and a router. For

each input token, the router selects a small subset of experts and combines their outputs according to the routing weights [2, 5, 8, 16]. We refer to an expert's model-defined identity within layer ℓ as its *logical expert ID*; experts with the same numeric ID in different layers have different weights and are therefore distinct model objects. For an invocation of layer ℓ at inference step t, let $\mathcal{A}_{\ell,t}$ denote the set of logical experts selected across all participating tokens. Because routing is input-dependent, $\mathcal{A}_{\ell,t}$ may vary across invocations of the same layer. This active-set variability is especially pronounced during prefill, where many prompt tokens are processed together and may collectively activate substantially more experts than a typical decoding step [4].

When the complete expert pool does not fit in accelerator HBM, expert offloading places part of the expert weights in host memory. We focus on *cache-based expert offloading*, which maintains a bounded expert cache in HBM and stages non-resident experts from host memory on demand [1, 4, 7, 25]. Let $\mathcal{H}_{\ell,t}$ denote the experts of layer ℓ currently resident in HBM, with $|\mathcal{H}_{\ell,t}| \leq C_\ell$, where C_ℓ denotes the expert-cache capacity assigned to that layer. A requested expert $e \in \mathcal{A}_{\ell,t} \cap \mathcal{H}_{\ell,t}$ is a cache hit, whereas $e \in \mathcal{A}_{\ell,t} \setminus \mathcal{H}_{\ell,t}$ incurs an expert cache miss and must be staged from host memory. If the cache is full, admitting a missing expert requires evicting or replacing an existing resident expert.

Cache-based offloading therefore makes expert residency and placement runtime-managed state. For each resident expert $e \in \mathcal{H}_{\ell,t}$, let $\pi_{\ell,t}(e)$ denote the device storage location assigned to its weight tensors at step t. Changes in $\mathcal{A}_{\ell,t}$ do not necessarily change placement: newly requested experts may already be resident. When routing does cause admission, eviction, or replacement, however, both the resident set $\mathcal{H}_{\ell,t}$ and the placement mapping $\pi_{\ell,t}$ may change across invocations of layer ℓ. Thus, a logical expert has a fixed model identity, while its HBM residency and device storage location are determined dynamically at runtime.

2.2 Graph Replay and Stable Storage Bindings

Graph replay reduces repeated host-side scheduling and kernel-launch overhead by capturing a recurring accelerator execution sequence and replaying it as a unit [17, 26].

Captured computation reuses both its operation structure and the storage bindings referenced by those operations. For the captured expert computation of layer ℓ, we denote these bindings by $\mathcal{B}_\ell$.

Graph replay does not require the contents of this storage to remain unchanged. Values in preallocated buffers may be updated between replays as long as the captured computation continues to access the same storage locations. For example, FlashInfer updates request-dependent scheduling metadata in preallocated device workspaces while preserving the workspace locations referenced by captured kernels [26].

Thus, graph replay can accommodate dynamic data behind stable storage bindings. It is a change to the bindings themselves that may require graph update or recapture [14].

2.3 Dynamic Expert Placement Meets Graph Replay

The conflict arises when captured computation directly consumes the device storage assignments maintained by an expert cache. Fix a layer ℓ and consider one of its logical experts e that is resident at two inference steps t and $t' > t$. If e is evicted after step t and later reloaded, it may occupy a different HBM location:

$$\pi_{\ell,t}(e) \neq \pi_{\ell,t'}(e).$$

Such placement changes are natural under a bounded expert cache, whose contents evolve with routing decisions.

Dynamic expert placement does not inherently conflict with graph replay. The conflict occurs when captured computation directly references the current storage location of a logical expert.

If captured expert computation directly binds e to its current HBM location, a change in $\pi_{\ell,t}(e)$ also changes the graph-visible binding $\mathcal{B}_\ell$. Dynamic expert caching may therefore require the resident set $\mathcal{H}_{\ell,t}$ and placement mapping $\pi_{\ell,t}$ to evolve, while conventional graph replay expects its graph-visible bindings $\mathcal{B}_\ell$ to remain reusable. Cache-based expert offloading and graph replay therefore impose different requirements on storage state:

$$(\mathcal{H}_{\ell,t}, \pi_{\ell,t}) \text{ may evolve,} \qquad \mathcal{B}_\ell \text{ remains replay-stable.}$$

The desired abstraction must allow changes in expert residency and placement without propagating those changes to the storage bindings observed by captured computation.

Several straightforward alternatives avoid one side of this conflict only by sacrificing another system objective. Keeping all experts permanently resident would stabilize their physical bindings but defeats offloading under a bounded HBM budget. Falling back to eager execution accommodates arbitrary placement changes but forfeits the host-overhead savings of graph replay. Updating or recapturing the graph after each placement change similarly reintroduces host intervention into the repeated inference path.

Therefore, our goal is to support dynamic expert replacement within a bounded HBM cache while preserving the graph-visible storage bindings required by graph replay.

3 Motivation

Section 2 shows that cache-based expert offloading can conflict with graph replay when routing-driven , per-layer expert placement changes alter the storage bindings used by captured computation. We next ask whether this conflict arises under realistic routing behavior. Three empirical questions determine the answer: (1) whether decode routing is sufficiently stable, (2) whether prefill working sets fit within the available per-layer capacity, and (3) whether

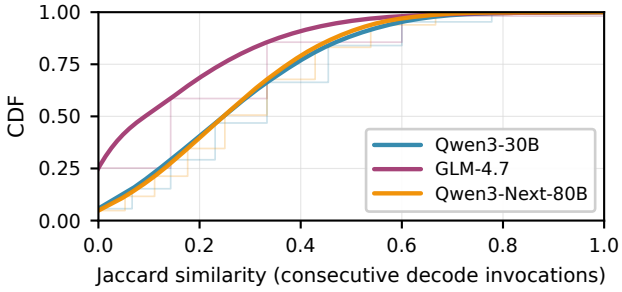

Figure 2. Decode active sets change rapidly. Jaccard-similarity CDFs across consecutive within-request, same-layer invocations; lower values indicate greater turnover. Steps are empirical; solid curves are monotone guides.

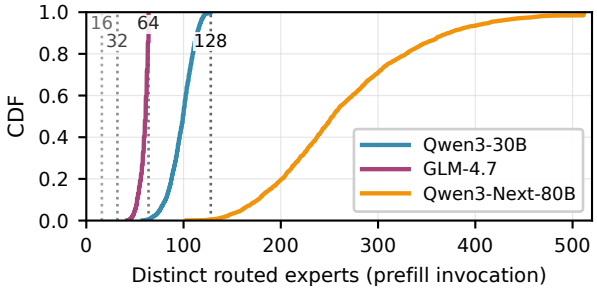

Figure 3. Prefill working sets exceed slot capacity. CDFs of distinct routed experts per layer invocation; vertical lines denote capacities of 16, 32, 64, and 128 slots. Every profiled invocation exceeds 32 slots.

the expert transfers required by bounded-capacity execution can be serialized without substantial overhead. We profile Qwen3-30B-A3B, GLM-4.7-Flash, and Qwen3-Next-80B-A3B-Instruct on one Ascend 910B2 under controlled, single-request serving (BF16, TP=1), using 200 ShareGPT requests, respectively, and 32 expert slots per managed layer.[1]

3.1 Decode Routing Forces Frequent Placement Changes

Graph replay repeatedly reuses captured expert computation, so a bounded expert cache would be sufficient if routed demand changed little between consecutive invocations of the same MoE layer. We quantify this temporal stability using Jaccard similarity, $J_{\ell,t} = |\mathcal{A}_{\ell,t} \cap \mathcal{A}_{\ell,t+1}|/|\mathcal{A}_{\ell,t} \cup \mathcal{A}_{\ell,t+1}|$, where lower values indicate greater routing turnover.

Figure 2 shows that such stability is weak. The median Jaccard similarity is only 0.333 for Qwen3, 0.143 for GLM, and 0.250 for Qwen3-Next. Routing turnover does not always cause a placement change because a newly requested expert may already be resident. Cache traces, however, show that expert misses account for 20.6%, 14.0%, and 38.2% of requested expert accesses, while 68.1%, 40.4%, and 93.9% of decode-layer invocations update at least one expert-to-storage mapping under the 32-slot configuration. If current placement is directly exposed to captured computation, these updates alter its storage bindings and require graph update or recapture, or force execution to bypass replay. Sparse activation therefore reduces per-token computation but does not provide placement stability across replays.

Implication. Dynamic expert placement must be decoupled from the stable storage bindings used by captured computation.

[1]Qwen3-30B-A3B has 48 MoE layers with 128 experts per layer and top-8 routing; GLM-4.7-Flash has 46 MoE layers with 64 experts per layer and top-4 routing; and Qwen3-Next-80B-A3B-Instruct has 48 MoE layers with 512 experts per layer and top-10 routing. Generation is limited to 128 output tokens for Qwen3 and GLM, and 32 output tokens for Qwen3-Next.

3.2 Prefill Working Sets Outgrow Bounded Slot Capacity

A cache large enough for the peak routed set could avoid replacement within an invocation. Figure 3, however, shows that a layer's prefill working set $|\mathcal{A}_{\ell,t}|$ routinely exceeds its assigned capacity C_ℓ. With 32 slots, every observed prefill-layer invocation overflows: median routed expert counts are 99, 61, and 252 for Qwen3, GLM, and Qwen3-Next, while the corresponding 95th percentiles are 117, 64, and 401.

Increasing capacity provides only partial relief. A 64-slot cache still overflows 99.3% of Qwen3 invocations and every Qwen3-Next invocation, while even 128 slots overflow 99.5% of Qwen3-Next invocations. These measurements use single-request execution, so the overflow arises from one invocation's routed demand rather than cross-request concurrency. Provisioning device storage for the peak routed set is also undesirable because HBM must accommodate the remaining model state and KV cache.

Implication. Supporting a layer working set larger than C_ℓ requires reusing that layer's bounded execution storage across multiple waves within one invocation.

3.3 Serial Expert Staging Would Dominate Per-Layer Execution

When a routed working set exceeds the available slots, its experts must be processed in multiple waves. A straightforward implementation would execute these waves serially: stage the experts for one wave, compute that wave, and only then stage the next. Figure 4 shows that such serialization would be expensive. Across the three models, aggregate measured expert-staging time is 2.36×, 3.13×, and 2.01× the corresponding MLP execution time. If fully exposed on the critical path, staging would account for approximately 70%, 76%, and 67% of the serialized stage-plus-MLP execution time.

Avoiding this transfer bottleneck requires staging the next wave while the current wave is still computing. This

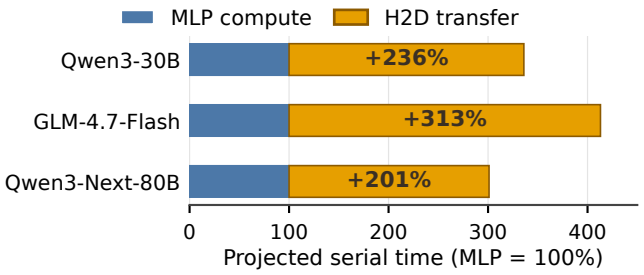

Figure 4. Serially exposed expert staging would dominate MLP execution. MLP execution is normalized to 100%; the appended bars show aggregate measured staging time. If fully serialized, staging is 2.01–3.13× the corresponding MLP execution time.

overlap, however, makes bounded storage reuse asynchronous: a storage location cannot be reassigned until its previous computation has finished, while a newly staged expert cannot be consumed until its transfer has completed. **Implication.** Bounded staging must overlap expert transfer with computation. It must also coordinate transfer readiness, computation completion, and storage reuse.

Together, these observations motivate three requirements for piecewise expert offloading: stable replay bindings under dynamic placement, bounded execution for oversized working sets, and safe transfer–compute overlap. Section 4 presents LatchMoE's design for these requirements.

4 Design

LatchMoE uses a capacity-adaptive piecewise architecture to reconcile dynamic expert placement with graph replay. It routes once and then selects between two expert-execution paths. When the routed set fits the persistent bank, LatchMoE replays captured expert computation against fixed storage bindings. When a prefill working set exceeds that capacity, it executes exact eager waves from fixed transient banks and reassembles their outputs before native composition. Both paths preserve the same routing semantics and versioned storage-lifecycle rules. The eager overflow path is an explicit execution mode, not a fallback from failed graph replay. The design must therefore preserve stable replay bindings, execute routed expert sets that exceed the available execution-bank capacity, and safely coordinate asynchronous storage reuse.

4.1 Execution Model and Invariants

Consider one managed MoE layer. Its native router produces a set of routed token–expert pairs $P = \{(i, e, j, w)\}$, where i is a token, e is a layer-local logical expert, j is its top-k position, and w is its routing weight. Let $E(P)$ denote the distinct experts in P. An execution bank has capacity C and a fixed set of device buffers $\mathcal{S}$ for expert-weight groups. Its bank-local addressing contract maps active logical experts to local slots. Let $\beta_r(e) = s$ denote the active bank-local binding consumed for expert e in wave r. The persistent replay path realizes β_r through a fixed indirection buffer $\mathcal{M}$; an overflow wave encodes bank-local slot IDs in its wave plan. In the evaluated layouts, the weight groups are W_{13} and W_2. The addresses, shapes, and types of the persistent buffers form the storage binding visible to captured expert computation.

A managed invocation follows the same logical path regardless of whether $E(P)$ fits in one bank; capacity determines the number and execution mode of its waves. First, a captured routing piece produces P once. The runtime inspects the complete $E(P)$, determines whether it fits, and constructs a capacity-bounded plan. A fitting invocation uses the persistent bank and its prebound captured expert piece. An overflowing prefill invocation instead materializes each E_r as a complete wave in one fixed transient bank and executes the wave through the eager expert path. Staging and expert execution repeat until every routed pair has executed; their outputs are then restored to the original pair positions and passed to the native composition path. With multiple transient banks, staging wave $r + 1$ may overlap computation of wave r.

Figure 5 separates these two expert paths. An expert execution instance is associated with one pre-established bank binding; the runtime never retargets one captured graph to a different allocation. The persistent expert piece therefore continues to reference its capture-time binding, while overflow execution selects among fixed transient banks in the eager wave path. In both cases the seam may change slot contents and the active binding β_r, but it does not reallocate the selected bank.

Correct execution rests on four invariants.

1. **Stable replay bindings.** The persistent bank's weight buffers $\mathcal{S}$ and indirection buffer $\mathcal{M}$ retain the same allocations across capture and replay. Once allocated, transient banks are reused in place, but are not rebound into the captured expert path.
2. **Publish after readiness.** An active binding $\beta_r(e) = s$ becomes consumable only after the weights of expert e are ready in slot s for the current ownership version.
3. **Reuse after completion.** A slot cannot start its next ownership version until computation using its current version has completed.
4. **Exact routed semantics.** Every pair in P executes exactly once with its original expert ID, top-k position, and routing weight, even when P is divided across waves.

Together, the first invariant makes persistent placement changes compatible with replay; the next two make asynchronous storage reuse safe on both paths; the last preserves native MoE computation under capacity pressure.

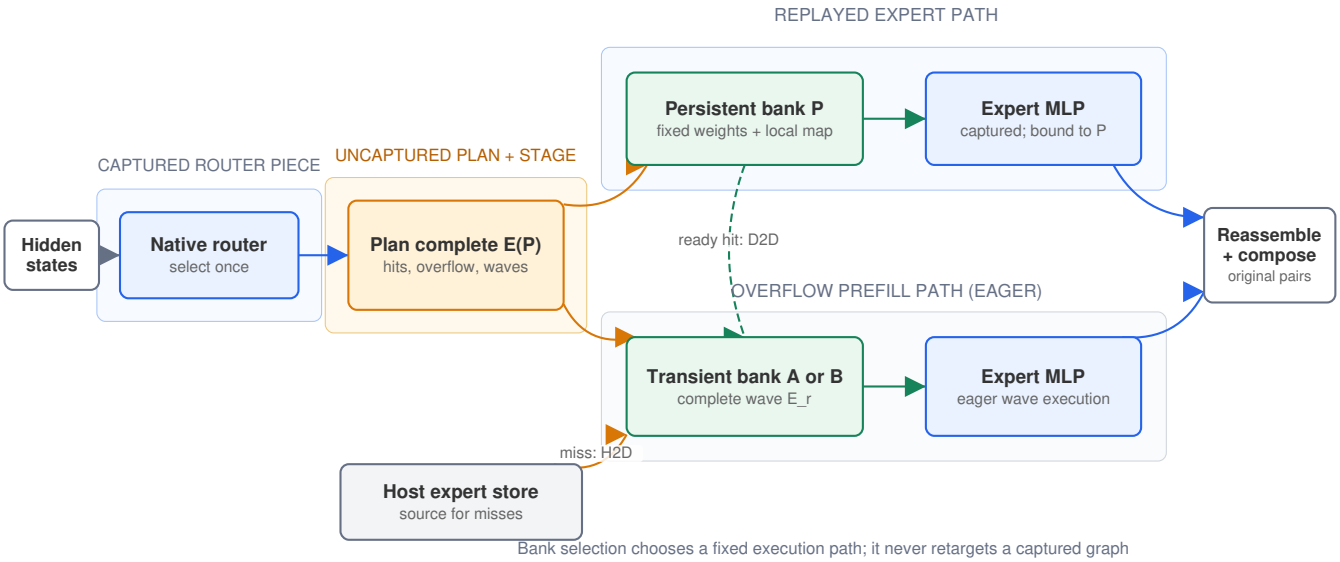

Figure 5. LatchMoE's route–plan–execute model. The persistent replay path is bound to one capture-time bank allocation. Overflow prefill uses complete waves in fixed transient banks rather than rebinding that captured piece. Both paths preserve the original routed pairs for one native composition.

4.2 Stable Replay Bindings and Bounded Storage

Expose routing without changing it. Staging cannot start until the active logical experts are known, but a monolithic fused MoE operation hides selection inside the captured operation. LatchMoE therefore relocates native selection into the first captured piece rather than predicting or reimplementing it. This piece invokes the model's native router with its configured parameters and emits the selected IDs and weights. The seam may inspect IDs to plan waves and residency, but it cannot add, drop, or rewrite routed entries. Both tensors continue unchanged to expert execution, so routing is evaluated once and retains the model's native decision.

Storage roles. Routed weights remain available in a host expert store. Each managed layer has a layer-private persistent bank that preserves expert residency across invocations, especially during decode. Overflow waves use transient banks as bounded scratch storage, so staging a large prefill working set need not evict the persistent working set. Compatible layers may share the transient-bank pool because their executions are serial, whereas persistent banks remain layer-private to retain temporal reuse.

The banks do not form one global slot namespace. The persistent bank and each transient bank expose separate fixed weight buffers and bank-local addressing; a plan selects one such pre-established binding and uses slot IDs local to that bank. The captured expert piece is bound only to the persistent bank selected at capture; choosing transient Bank A or B selects the eager wave path against that fixed bank rather than rebinding the captured piece. Reassignment changes buffer contents, ownership metadata, and active indirection values, but not bank allocations. Logical IDs are also layer-local: equal numeric IDs in different layers do not imply shared weights or residency.

The shared expert, when present, follows a separate lane. Because it is used by every applicable token, LatchMoE leaves it device-resident and executes it once per invocation. It neither consumes routed-slot capacity nor participates in victim selection or wave partitioning.

Stage, then publish. After the router piece completes, the runtime first inspects the complete routed set $E(P)$ to determine persistent hits, detect overflow, partition waves, and select their execution banks. Materialization is then local to the current route or wave rather than to all of $E(P)$. On the fitting replay path, a ready hit retains its persistent slot; for a miss, the runtime selects a reassignable persistent slot, removes its previous owner from the authoritative residency metadata, advances its ownership version, and starts the H2D weight transfer.

On the overflow path, every expert in E_r is assigned a bank-local transient slot so that one bank supplies the complete wave. A persistent hit is copied device-to-device into that slot, whereas a miss is copied from the host store. The transient slot's ownership version advances in either case. Only after all required transfer dependencies are satisfied and each slot still matches its issued owner and version does

the runtime mark the binding ready and publish β_r, making its bank-local slot IDs consumable by expert execution.

On the replay path, the device indirection buffer is a consumption interface, not the authoritative residency directory. Its entries outside the active route are unspecified and may retain values from an earlier invocation because captured computation cannot reach them. On the overflow path, β_r is instead encoded by the current wave's bank-local slot IDs. Thus, for either interface, only bindings reachable by the current expert operation must be valid; inactive entries are semantically irrelevant. Before execution, every active $\beta_r(e) = s$ must name a ready slot with a matching owner and version. Publishing an active binding before readiness could expose partially transferred weights; accepting completion for another ownership interval could publish the wrong expert. Hits and misses nevertheless leave the seam through the same bank-local storage contract.

4.3 Exact Execution beyond Slot Capacity

Stable bindings alone do not handle an invocation whose routed working set exceeds C. Let $E = E(P)$. When $|E| > C$, LatchMoE partitions the expert set into $E_1, \ldots, E_m$ and derives each wave's routed pairs:

$$\bigcup_{r=1}^{m} E_r = E, \qquad E_r \cap E_q = \varnothing \ (r \neq q), \qquad |E_r| \leq C, \qquad P_r = \{(i, e, j, w) \in P \mid e \in E_r\}. \tag{1}$$

Thus, all pairs for one expert remain in the same wave, and every pair in P belongs to exactly one P_r. The partitioner may change expert execution order, but not pair membership, top-k positions, or routing weights. This is tiling the expert dimension rather than approximating routing semantics.

Once overflow selects multi-wave execution, each E_r is materialized in full in one transient bank. Ready experts are copied from the persistent bank and misses from the host store; expert computation never mixes weights from both banks within one wave. This deliberate D2D copy preserves one coherent bank-local binding and leaves persistent cache residency undisturbed. Since each wave contains at most C experts, bounded scratch storage can be reused to cover the larger routed set.

Each wave computes only its assigned pairs and records their original pair positions. After all waves finish, LatchMoE reassembles the routed contributions at those positions and invokes the model's native composition path with the original weights and output shape. The shared lane, if present, is computed once and combined only after the routed contribution is complete. Consequently, wave boundaries do not duplicate shared work or change model-specific routed scaling, reduction, or postprocessing. Coverage validation treats a missing or duplicate pair as an execution error.

For example, with $C = 2$, the routed set $\{e_2, e_5, e_8\}$ requires at least two waves. Every expert's routed pairs remain intact within one wave and are restored to their original positions before native composition.

4.4 Safe Transfer–Compute Overlap

Serially staging every wave would expose transfer latency. LatchMoE overlaps the transfer of wave $k + 1$ into Bank B with computation of wave k from Bank A, as shown in Figure 6(a). Bank A can then receive wave $k + 2$, but only after wave k has stopped reading it. The same rule applies to individual persistent slots and to whole transient banks.

Each slot carries a fixed device-buffer address and mutable (*owner*, *version*, *state*) metadata. Its state is one of *empty*, *loading*, *ready*, or *computing*. Assignment of expert e to slot s advances the version to g, enters loading, and returns a readiness event for the lease $\lambda = (s, e, g)$. A loading slot is never selected for another assignment. When the transfer-completion dependency is satisfied, the runtime validates λ, enters ready, and may publish the active binding. Expert computation starts only after that publication, acquires the ready lease, and moves it to computing. Here, *computing* denotes one outstanding expert-operation lease for the current slot version; that operation may contain routed pairs from several batched requests. A subsequent operation cannot acquire or reassign the slot until the completion event returns it to ready. The slot does not become empty merely because one use completes, since retaining its owner enables a future cache hit.

The protocol imposes the following happens-before relations:

$$\begin{aligned} &\text{ready}(s, e, g) \prec \text{publish}(\beta_r(e) = s) \prec \text{compute}(s, e, g), \\ &\text{done}(s, e, g) \prec \text{assign}(s, e', g + 1). \end{aligned} \tag{2}$$

The first chain prevents incomplete or unpublished weights from being consumed. The second prevents a new transfer from overwriting weights still consumed by a kernel. Normal scheduling therefore reassigns neither loading nor computing storage. Owner/version checks additionally reject a delayed event from an earlier ownership interval instead of allowing it to authorize publication or reuse.

At bank granularity, the same lease rule gives a simple pipeline. Transfer into Bank B may proceed while Bank A is computing because the banks are disjoint. Reusing Bank A for wave $k + 2$ waits specifically for the completion event of wave k; it need not serialize on unrelated work in Bank B. This coordination exposes exactly the dependencies needed for safety while leaving independent transfer and compute eligible to overlap.

4.5 Design Boundary

LatchMoE separates execution correctness from placement quality. A placement policy may choose residency, wave membership, and execution order only over the complete

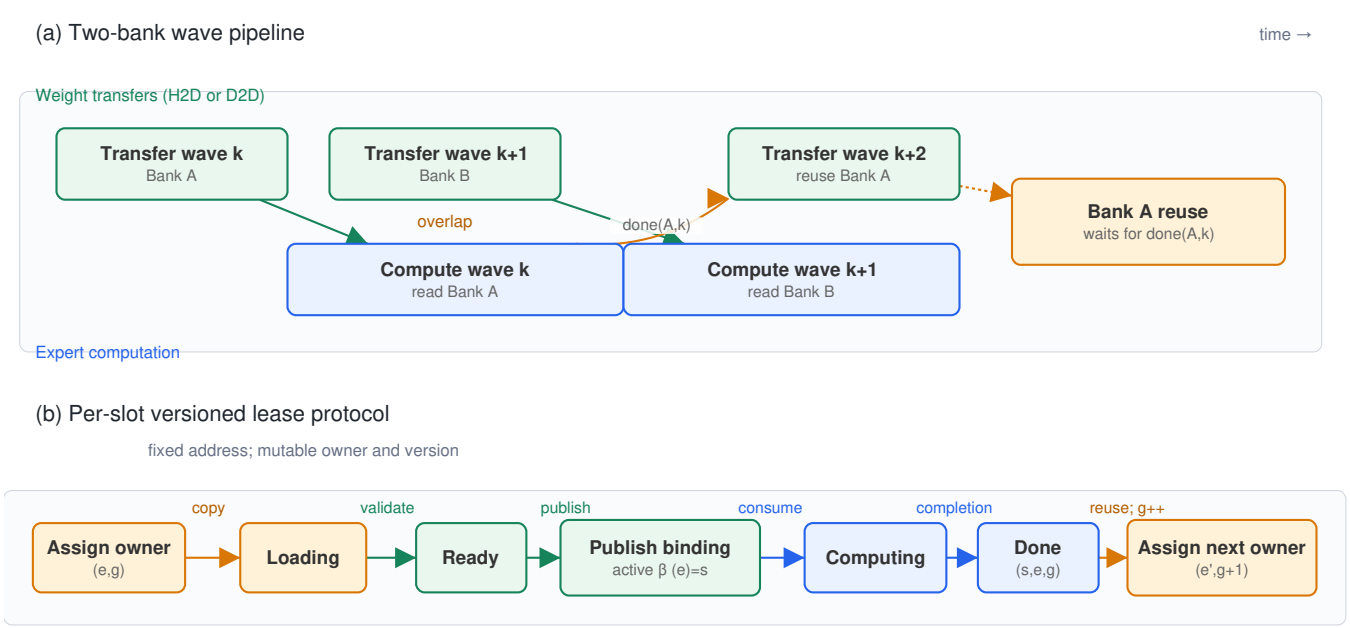

Figure 6. Versioned leases order asynchronous staging and bounded reuse. (a) Two banks pipeline consecutive waves; a bank's next transfer waits for its previous compute completion. (b) The active binding is published only after transfer readiness and consumed only after publication; reassignment waits until no computation holds the current lease.

routed expert set; it cannot add or remove routed experts or alter capacity constraints, publication, or lease rules. The split interface is enabled only for supported router, shared-expert, and output contracts, while unsupported contracts are rejected rather than approximated. Concrete combinations of model and runtime still require semantic and replay qualification, which Section 6 reports.

5 Implementation

We implement LatchMoE as a Python package that registers through vLLM's general plugin entry point. The package integrates with a hook-enabled vLLM-Ascend runtime; the upstream request scheduler and OpenAI-compatible serving API remain unchanged. The host runtime and the Ascend platform plugin are pinned as a compatible stack because the staging boundary changes the internal fused-MoE execution contract rather than the public serving interface.

5.1 Graph integration

The integration decomposes a managed MoE layer into a captured router, the custom `vllm::moe_offload_stage` splitting operation, and a capacity-selected expert path. The splitting operation returns control to the host after routing so that LatchMoE can stage the active experts. A fitting invocation continues into the captured expert MLP, which reads persistent slot weights and the logical-to-slot buffer. An overflowing prefill instead invokes the eager wave executor against transient stage banks. The runtime locks the persistent buffers' data pointers at capture and validates them before replay.

Graph-compatible runs use PIECEWISE ACLGraph. The benchmark validator requires explicit capture and replay records for graph-designated pieces and rejects forced eager execution or unintended eager fallback of those pieces, graph errors, address changes, stale H2D completion, and slot-version violations. The planned eager overflow path is not classified as fallback. Disabling LatchMoE restores the host runtime's null hooks. The launcher also rejects simultaneous use of the native weight-offload path and LatchMoE because two independent owners of expert storage would violate the slot lifecycle.

5.2 Loading and memory management

CPU-first loading creates managed expert tensors in host memory during model initialization. The host store keeps one contiguous W_{13} tensor and one contiguous W_2 tensor per layer and exposes per-expert views to the transfer engine. It checks shape, dtype, stride, device, layer coverage,

and expert coverage before releasing the original device expert banks. Host pinning is used when the runtime supports it and reports failures explicitly.

Automatic configuration derives the resident-layer selection and slot count from the requested offload amount, physical HBM, a KV-cache reserve, and a cap on the fraction of HBM assigned to slot storage. Explicit overrides remain available for controlled sensitivity experiments. The memory ledger accounts separately for the host store, persistent slots, transient prefill stage banks, mapping tensors, optional full-layer staging storage, and any original weights that remain allocated.

5.3 Transfers and lifecycle enforcement

The transfer engine maintains a dedicated H2D stream. It validates layout compatibility before copying and batches consecutive host and slot slices when their storage is contiguous. Each asynchronous load returns a readiness object that contains the NPU event and the exact slot leases made consumable by that event. The consumer installs the event dependency before the object marks the leases ready. The runtime then publishes the active persistent mapping or wave-local binding.

Compute protection uses an event recorded after the expert operation. Slots remain in the computing state until the event completes and every recorded lease still matches its current owner and version. Shutdown drains pending transfer and compute work, closes profiling writers, releases managed storage, and emits a release acknowledgement for the benchmark harness.

5.4 Wave execution and profiling

The prefill path supports a configurable number of stage banks and prefetch depth. The qualified configuration uses two stage banks and depth one. Wave plans carry the original pair offsets needed for one native recombination, and strict validation rejects duplicate or missing coverage. A controlled evaluation configuration may replace bounded waves with explicit full-layer staging; it is not an error-recovery path. NPU, ACL, memory, address, semantic, and lifecycle failures are not swallowed by either execution mode.

Profiling records JSONL events for layer registration, slot ownership, transfers, mapping publication, wave staging and compute, graph replay issue, memory accounting, and service release. Benchmark manifests bind each run to the runtime revisions, model and workload digests, device, command line, environment, graph mode, and raw client/server artifacts. These records support the fail-closed checks used in Section 6.

6 Evaluation

We ask six questions. **Q1** quantifies the deployability trade-off against same-stack baselines. **Q2** isolates PIECEWISE Graph execution from the fixed-slot mechanism. **Q3** measures capacity-bounded waves against graph-compatible full-layer staging. **Q4** isolates transfer–compute overlap. **Q5** tests sensitivity to slot capacity and bounded request overlap. **Q6** evaluates model-contract generality and accounts for resources and unsupported boundaries.

6.1 Experimental Methodology

Platform and software. Experiments use one Ascend 910B2 with 65,536 MiB HBM and tensor parallelism one, hosted by a 192-core Kunpeng-920 server with 2.0 TiB DRAM. Models use BF16 weights. The managed stack pins PyTorch 2.10.0, torch-npu 2.10.0.post2, vLLM commit `ad7125a`, and vLLM-Ascend commit `4806367`. Each run manifest records the complete command, model and dataset digests, source-tree digest, environment, device, graph mode, and service-release custody.

Workloads and controls. The cost-anchor, overlap, and capacity campaigns use a frozen 64-request prefill-heavy ShareGPT bucket with 1034–1979prompt tokens (median 1394), generate 32 tokens at temperature zero and seed 42, disable prefix caching, and set both concurrency controls to one. The full-layer/multi-wave campaign uses a separate frozen 200-request ShareGPT manifest and 128 output tokens. Every formal unit starts a fresh service. Counterordered comparisons use three start pairs; the paired estimator is the exponential of the mean, across starts, of the median per-request log latency ratio. We report a 10,000-replicate hierarchical bootstrap with seed 20260823: 95% intervals for TTFT effects and 90% intervals for the preregistered TPOT equivalence test (margin ±5%). For the Qwen3 performance campaigns, the 14-GiB AutoConfig manages 12 of the model's 48 MoE layers (one layer in each four-layer group); all four baseline arms and Q3–Q5 use this identical layer selection. Q6 is a separate capability qualification and exercises the model-specific layer counts shown in Table 2.

The c1 Graph qualification uses the frozen standard-v1 ShareGPT, HumanEval, and GSM8K buckets (200, 164, and 200 requests), generates four tokens, and holds the same Qwen3 model, 64 slots, 14-GiB host target, 2-GiB KV cache, request order, and sampling controls across Eager and PIECEWISE Graph modes. Both concurrency controls are one; each bucket uses three counterordered fresh-start pairs. LongBench is retained separately as an adverse boundary because all 200 frozen requests exceed the current 8,192-token context configuration.

Baseline roles. Full-resident vLLM-Ascend is the HBM and latency cost anchor, not a memory-matched competitor. Native weight prefetch and legacy layered offload are same-stack deployment competitors, admitted to performance comparison only if they pass exact-token parity. `Slot+Eager` is the matched causal control for Graph execution: it retains the same slot bank, host store, wave plan, and lifecycle protocol as `Slot+PIECEWISE`. Full-layer staging is the matched design alternative to the complete bounded-wave path. Because overflow also selects the eager wave executor, Q3 is not a one-factor isolation of partitioning alone. Serial staging is the one-factor ablation for overlap.

Fail-closed gates. A reportable unit must complete all requests, match complete output-token arrays by request ID, retain fixed slot and map addresses, publish no stale transfer, violate no owner/version lease, avoid NPU/ACL errors, emit a release acknowledgment, and retain raw artifacts. Graph arms must additionally capture and replay every graph-designated PIECEWISE ACLGraph component without unintended fallback; the explicitly selected eager overflow path is part of the execution contract and is not counted as fallback. Eager controls must record Graph as disabled. A completed arm that fails exact-token comparison is retained but excluded from latency comparison.

6.2 Q1: What is the deployability trade-off?

We run three counterordered starts of four ACLGraph arms with an explicit 512-MiB KV cache and 32requests per unit: full-resident vLLM-Ascend, its native weight-prefetch path, the legacy layered offload path, and LatchMoE with a 14-GiB target. Full residency and LatchMoE each pass 96/96 exact requests. Native prefetch and legacy layered offload both complete 96/96 requests and capture/replay the graph, but respectively differ from the oracle on 30 and 30 request-token arrays across their three starts. They remain visible feasibility cells but are excluded from performance comparison.

Full residency is a cost anchor rather than a memory-matched offload baseline. Its repeat-median TTFT p50 and TPOT p50 are 179.13 ms and 13.19 ms/token, versus 662.22 ms and 54.50 ms/token for LatchMoE. The paired TTFT effect is +261.28% (95% CI [+240.09%, +277.32%]); the TPOT effect is +309.51% (95% CI [+299.13%, +319.38%]). In return, the median sampled HBM peak falls from 97% to 80%.

Table 1 makes the trade-off explicit. LatchMoE stores 13.50 GiB in the host expert store and 3.38 GiB in persistent slots, plus a 0.56 GiB prefill stage bank; the profile records 292.05 GiB of H2D traffic. The measured 17-percentage-point HBM reduction costs 261.28% higher TTFT p50 and 309.51% higher TPOT p50. The result is therefore single-card deployability with an explicit latency cost, not a universal speedup. This trade-off matters for larger tuples:

Table 1. Same-scope full-resident cost anchor for Qwen3-30B-A3B: three fresh starts, 32 requests per arm, 512-MiB KV cache, and disabled prefix caching. HBM reclamation is the difference in median sampled peak utilization; stage wait is reported from the profile, not substituted for end-to-end latency.

Metric	Full resident	LatchMoE
TTFT p50 (ms)	179.13	662.22
TPOT p50 (ms/token)	13.19	54.50
Output throughput (tok/s)	52.94	13.23
Sampled peak HBM (%)	97	80
Host store (GiB)	0	13.50
Slot HBM (GiB)	0	3.38
Prefill stage HBM (GiB)	0	0.56
H2D traffic (GiB)	0	292.05
Prefill stage wait (ms)	0	257.11
Decode ready wait (ms)	0	95.05

Derived cost: LatchMoE reclaims 17 percentage points of sampled peak HBM, while adding +261.28% TTFT-p50 and +309.51% TPOT-p50 relative to full residency.

Qwen3-Next's native full-resident end-to-end path cannot materialize on the evaluated 64-GiB device, whereas its bounded LatchMoE qualification completes (Q6).

6.3 Q2: Does Graph preserve the matched slot path?

The matched comparison changes only the execution mode from `Slot+Eager` to `Slot+PIECEWISE`. Across ShareGPT, HumanEval, and GSM8K, 18 fresh units form nine counterordered pairs and complete 3,384 requests. All 13,536 output token IDs match within pair, and no cross-start oracle drift is observed. All nine Graph units configure PIECEWISE, reach the offload staging seam, and record actual ACLGraph replay; none records an unintended fallback of a graph-designated piece or an NPU/ACL failure. Thus Graph replay preserves the evaluated fixed-slot path at c1 across the three executable frozen buckets.

This four-token campaign is a semantic qualification, not a latency or task accuracy result. LongBench remains an explicit negative cell: all 200 frozen requests exceed the 8,192-token context/KV configuration and are neither trimmed nor replaced. Q5 separately reports the concurrency boundary.

6.4 Q3: Do bounded waves improve full-layer staging?

The graph-compatible full-layer arm materializes the layer working set before compute; the multi-wave arm instead uses bounded transient banks and the explicit eager wave executor to execute and recombine routed work. All model, layer-selection, workload, and serving controls remain fixed, but the bounded design necessarily changes both staging granularity and the overflow expert path. Q3 therefore

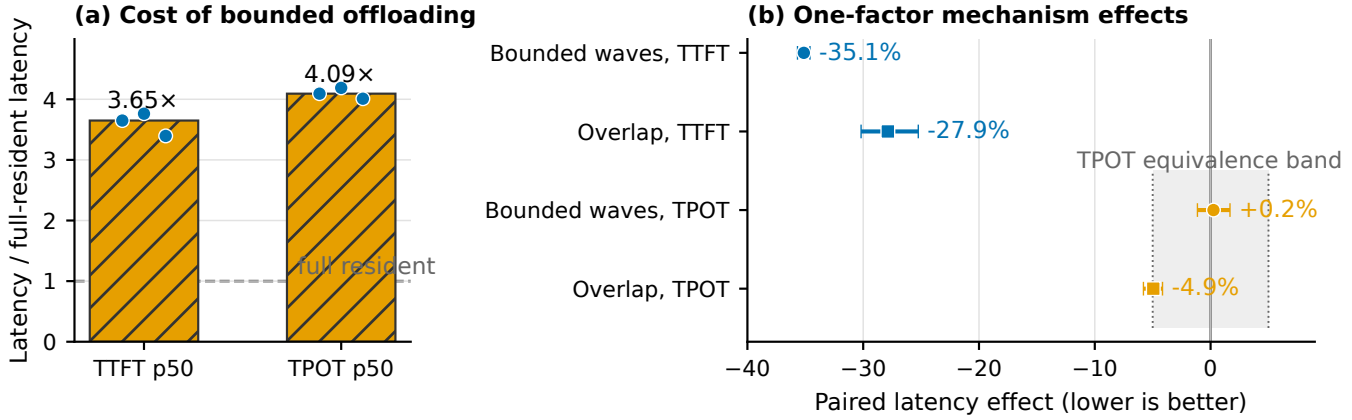

Figure 7. Bounded offloading has an explicit full-resident cost, while its two prefill mechanisms reduce exposed latency. (a) Bars are medians and points are three fresh exact-token, counterordered starts (96 requests per arm); the full-resident reference is 1.0. Native-prefetch and legacy-offload arms are omitted because they fail the exact-token comparability gate. (b) Bounded waves compare multi-wave against full-layer staging (200 requests per unit); overlap compares prefetch depth one against serial depth zero (64 requests per unit). Each contrast uses three counterordered pairs. Points are paired log-ratio estimators; whiskers are 95% CIs for TTFT and 90% CIs for TPOT. Lower is better. All experiments use Qwen3-30B-A3B, one Ascend 910B2, and concurrency one.

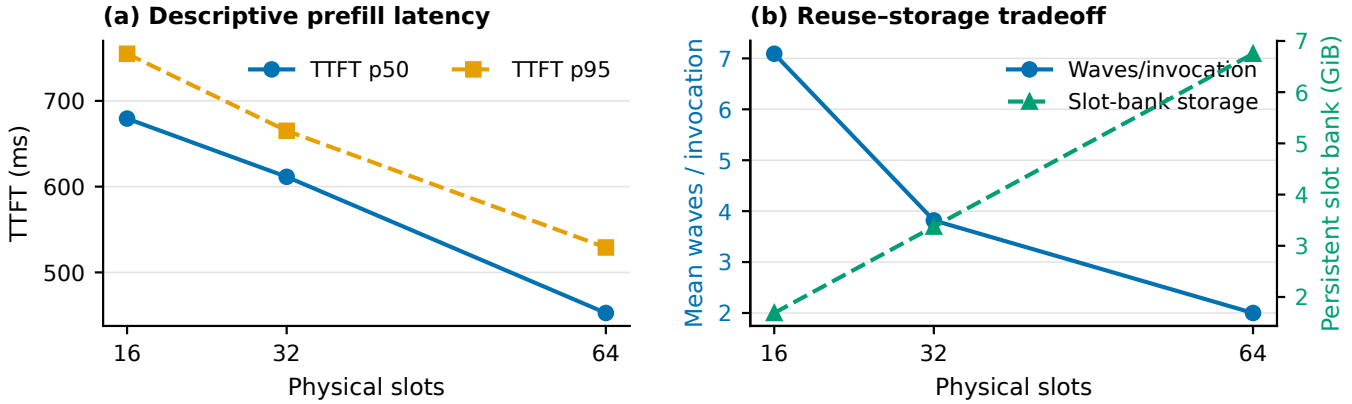

Figure 8. Online slot-capacity sensitivity for Qwen3-30B-A3B with one fresh start per point. All other serving controls are fixed, including a 14-GiB offload target, 512-MiB KV cache, 32 requests, and concurrency one on one Ascend 910B2 with BF16 and tensor parallelism one; prefix caching is disabled. More slots trade persistent HBM for fewer waves; latency values are descriptive.

evaluates the complete bounded-wave design rather than attributing its effect to partitioning alone. Three AB/BA/AB pairs each serve 200 requests. All six units complete 1200 requests, match all 153600 output token IDs and preserve the configured piecewise execution contract. Graph-designated pieces capture and replay as required, while overflow expert waves execute through the explicit eager path; no unit records unintended fallback or a lifecycle error.

TTFT changes by -35.13% with a 95% CI of [-35.67%, -34.60%]. As a conventional summary, the median across starts of TTFT p50 falls by 35.21%, and TTFT p95 falls by 22.96%. TPOT's paired effect is 0.25% with a 90% CI of [-1.14%, 1.68%], entirely within the preregistered ±5% equivalence margin. The supported result is therefore a prefill-path TTFT reduction without a material TPOT change under this contract.

6.5 Q4: Does overlap contribute causally?

We next hold the wave plan, two stage banks, graph mode, slot capacity, workload, seeds, and source digest fixed, changing only prefetch depth from zero (serial staging) to one (overlap). Three AB/BA/AB pairs serve 64 requests per unit; all 384 requests pass exact-token, graph, lifecycle, and release gates. Overlap changes TTFT by -27.87%

(95% CI [-30.18%, -25.24%]); the interval is entirely below zero and passes the preregistered causal-benefit gate. TPOT changes by -4.95% (90% CI [-5.80%, -4.15%]). Its equivalence gate does *not* pass because the lower bound extends beyond −5%; we report the measured decrease rather than calling TPOT unchanged. This one-factor experiment supports overlap's causal contribution, while Q3 measures the complete multi-wave path.

6.6 Q5: How robust is the slot contract?

We sweep 16, 32, and 64 physical slots with the same model, 14-GiB target, 512-MiB KV cache, first 32 manifest requests, and runtime source digest. Each point is one fresh start and passes exact-token, graph, capacity, lifecycle, and release gates; latency is therefore descriptive rather than an uncertainty claim. Figure 8 reports the resulting TTFT, average wave count, and persistent-slot storage. From 16 to 64 slots, TTFT p50 falls from 679.36 to 452.67 ms (33.37%), and p95 falls from 754.92 to 529.06 ms (29.92%). Total waves fall from 2808 to 792 and H2D volume from 318.5 to 173.8 GiB, while the persistent slot bank grows from 1.69 to 6.75 GiB and sampled peak HBM rises from 77% to 87%. This point quantifies the intended reuse–storage tradeoff; it is not a cross-workload scaling claim.

Capacity pressure also has a lifecycle dimension once requests overlap. In a separate bounded ShareGPT stress test, concurrency two and four both trigger real slot reuse and complete with exact outputs, monotonically increasing slot generations, H2D-ready publication before consumption, and balanced compute leases. This result exercises the ownership protocol beyond the c1 performance campaigns; because it is a targeted stress test rather than an offered-load sweep, it does not establish throughput or tail-latency scaling. The wider `standard_v1` qualification at concurrency eight exposes paired exact-token exceptions in ShareGPT, HumanEval, and GSM8K. We retain that negative result and admit none of its latency measurements; high-load correctness and performance remain outside the supported boundary.

6.7 Q6: Does the interface generalize across model contracts?

Table 2 covers routed-only Qwen3-30B-A3B, shared-expert GLM-4.7-Flash, and shared-expert Qwen3-Next-80B-A3B-Instruct. Each tuple is checked against native routing and layer boundaries, a native end-to-end oracle where feasible, eager-seam parity, exact output tokens, capture and replay of graph-designated pieces, zero unintended eager fallback, explicit overflow-path composition, decode cache churn, H2D ordering, and release. Every captured slot/map address validates at replay, every managed layer releases its original expert bank, and the online profiles exercise both overflow waves and H2D leases. LatchMoE therefore preserves the evaluated routing and combination contracts rather than merely making the models runnable.

This is capability rather than performance generalization. Each model uses one short request in each end-to-end mode plus a one-token overflow stress request. Qwen3 and GLM compare native, LatchMoE-eager, and LatchMoE-graph outputs. Qwen3-Next cannot materialize its native full-resident end-to-end path on this device, so native semantics are checked at its router and layer boundary and the end-to-end gate compares eager seam with Graph. We make no cross-model latency claim.

Resource costs and boundaries. Table 3 accounts for each model's graph-qualified 32-slot run. Host storage scales with the managed expert pool, while device storage depends on expert shape, shared residency, and stage banks rather than total expert count alone. Slot/stage storage is 13.5/0.56 GiB for Qwen3, 25.9/0 GiB for GLM, and 9.0/0.38 GiB for Qwen3-Next; mapping buffers are at most 0.19 MiB. Host-control time is 10.06 s, 0.073 s, and 44.85 s, respectively. These workload-specific values are costs, not a cross-model ranking. Qualification profiles record every original managed bank release; formal performance and Motivation services additionally retain a service-level release acknowledgment. The capability preflight rejects unknown router adapters, fused shared/routed ownership, quantized expert layouts, multi-NPU execution, and shared-expert overlap before capture. These are retained system boundaries: the evaluation establishes a single-NPU BF16 data plane, not silent fallback or support for unqualified tuples.

7 Discussion and Limitations

7.1 Stable storage is the reusable interface

Dynamic identity and graph replay conflict only when identity changes alter graph-visible allocation or execution structure. A fixed storage interface moves dynamism into contents and metadata updated at a controlled boundary. This principle may apply to other bounded working sets with a legal staging boundary and enforceable publication/reuse order; our evidence covers only the implemented MoE path.

Address stability does not prevent stale publication or eviction; owner/version checks and stream dependencies enforce temporal correctness. Static residency sacrifices bounded offload, pointer rebinding requires graph recapture, and falling a graph-designated piece back to eager pays dispatch overhead. LatchMoE fixes slots and stages replacement; its seam and fused operator are hardware-specific.

7.2 Control policy is outside the data plane

LatchMoE accepts an ordered placement plan but constrains it to the actual routed expert set. This boundary lets future

Table 2. Three-model semantic and graph qualification. $L/E/S$ denotes managed MoE layers, routed experts per layer, and resident shared experts. Router/boundary counts are exact native comparisons; locks/replays are address locks and observed graph replays; overflow/H2D counts exercise the explicit eager wave path and transfer leases. Every row passes all 11 gates, including exact end-to-end tokens and zero unintended fallback of graph-designated pieces.

Model	$L/E/S$	top-k	Router	Boundary	Locks/replays	Overflow/H2D	Result
Qwen3-30B-A3B	48/128/0	8	192	192	48/97	144/239	Pass
GLM-4.7-Flash	46/64/1	4	184	184	46/94	92/226	Pass
Qwen3-Next-80B-A3B	48/512/1	10	192	192	48/97	144/240	Pass

Table 3. Audited resources in each model's graph-qualified LatchMoE run. Device storage sums the 32-slot bank, prefill stage banks, and resident shared-expert weights; H2D is the profiled transfer volume for that qualification workload. Original managed expert banks are released in every row.

Model	Host (GiB)	Device (GiB)	H2D (GiB)	Replays
Qwen3-30B-A3B	54.0	14.1	65.5	97
GLM-4.7-Flash	51.8	26.7	22.3	94
Qwen3-Next-80B-A3B	144.0	9.7	152.6	97

caching or prediction policies reuse the fixed slots, transfer engine, wave executor, and graph checks. The current evaluation does not compare placement policies and should not be read as evidence that the default ordering is optimal. Such a study requires a shared trace, identical storage capacity, and policy-specific hit, transfer, and latency measurements on the same data plane.

7.3 Scope of the current evidence

The semantic and graph qualification covers three unquantized BF16 models on one Ascend 910B2 with tensor parallelism one. It validates native router and layer-boundary parity, exact end-to-end tokens, replay-visible addresses, overflow composition, lease/release records, and zero unintended eager fallback of graph-designated pieces for the three reported capability tuples. Performance experiments are narrower: Qwen3-30B-A3B, a 14-GiB host-offload target, serialized requests, `max_num_seqs=1`, and disabled prefix caching. The matched result shows lower TTFT for multi-wave than full-layer staging under that contract; the one-factor campaign isolates an overlap contribution. The full-resident anchor also exposes the absolute cost of host-backed replacement. Native prefetch and legacy layered offload complete graph-mode requests but fail the exact-token comparability gate, so we retain them without using their latency. Consequently, the reported performance evidence is an internal mechanism comparison against graph-compatible full-layer staging, not a superiority claim over published offloading systems. The full-resident cost anchor is therefore central to the deployment story: the measured 17-percentage-point reduction in sampled peak HBM is purchased with a 261.28% TTFT-p50 and 309.51% TPOT-p50 penalty in this scope. We state this as a latency cost for single-card deployability, not as a universal speedup. The slot sweep has one start per point and is descriptive. None of these results establishes a general decode-speedup or multi-request serving claim. A separate bounded c2/c4 stress test exercises real slot reuse, monotonic generations, H2D readiness, and balanced compute leases with exact outputs; it is lifecycle evidence rather than a throughput or tail-latency result. At c8, the broader frozen workload exposes paired exact-token exceptions, so we do not admit its latency measurements or claim high-load correctness.

Higher concurrency permits more overlapping request leases, prefix-cache reuse adds hits across the staging seam, multi-device execution adds distributed ownership, and quantization changes layouts and operator support. Their performance effects remain outside the qualified single-request campaigns.

8 Related Work

Sparse MoE inference. Sparsely gated MoE models increase parameter capacity while activating a subset of experts for each token [3, 5, 15, 16]. Systems for large MoE models also combine routing with model and expert parallelism [11, 15]. LatchMoE does not change the model, router, or expert computation. It addresses the runtime storage interface used when expert weights cannot remain fully resident on one accelerator.

Expert offloading and scheduling. Expert-offloading systems reduce host-to-device movement or its exposed cost through caching, prediction, prefetching, mixed precision, fine-grained placement, and pipelined execution [1, 4, 7, 9, 10, 19, 20, 25]. Recent CPU–GPU hybrid serving also stream-loads MoE prefill weights and overlaps CPU and GPU work, while FlexGen coordinates heterogeneous placement for dense models [18, 23]. These systems decide which work or weights to place, retain, and move. LatchMoE addresses an orthogonal execution constraint: changing a cache decision must not change the weight storage bound to replayed expert computation. Its placement

interface therefore feeds fixed slots and versioned leases instead of becoming part of the captured ABI.

Graph-based inference execution. Nimble compiles dynamic neural-network execution [17], while FlashInfer provides graph-oriented mechanisms for customizable LLM inference [26]. GraCE uses compiler transformations to increase graph coverage [6]. vTensor decouples computation from physical-page management while exposing a continuous virtual address [24], while ECHO makes dynamic KV-cache eviction and recall graph-friendly [12]. LatchMoE instead manages executable weights selected by a captured router and ordered against transfer and MLP computation. At an eager seam, capacity-fitting work exposes a fixed slot/map ABI to the downstream captured MLP; oversized prefills instead select exact eager waves rather than rebinding that graph.

Positioning. LatchMoE joins dynamic placement with graph replay by separating logical ownership from graph-visible allocation and checking each ownership interval as a lease. UVA alone supplies neither bounded HBM replacement nor ready-before-publication and release-before-reuse ordering.

9 Conclusion

LatchMoE combines two capacity-selected expert paths: fitting routed work replays captured expert computation from persistent, address-stable slots, whereas oversized prefills execute exact eager waves from bounded transient banks. Both paths order each ownership interval with a versioned lease. This piecewise contract passes semantic, graph, and lifecycle qualification across three materially different MoE models without unintended fallback of graph-designated pieces. On the scoped single-NPU, concurrency-one Qwen3 workload, bounded waves change paired TTFT by -35.13% in the matched full-layer comparison, and isolated overlap changes it by -27.87%; broader concurrency, quantization, and multi-NPU claims remain outside the qualified boundary. These reductions are a capacity–latency tradeoff rather than a universal speedup: against full residency in Q1, the same workload lowers median sampled HBM peak from 97% to 80% while increasing TTFT and TPOT by 261.28% and 309.51%, respectively.

Acknowledgments

OpenAI Codex was used to inspect and modify code, design and run experiments, analyze results, generate figure and table code, check citations, and assist with manuscript structuring, drafting, and editing. The authors reviewed the resulting code, artifacts, analyses, and text and take responsibility for the work.

References

[1] Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E. Gonzalez, Matei Zaharia, and Ion Stoica. 2025. MoE-Lightning: High-Throughput MoE Inference on Memory-constrained GPUs. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS 2025)*. Association for Computing Machinery, 715–730. https://doi.org/10.1145/3669940.3707267

[2] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 1280–1297. https://doi.org/10.18653/v1/2024.acl-long.70

[3] Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten P. Bosma, Zongwei Zhou, Tao Wang, Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc V. Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. 2022. GLaM: Efficient Scaling of Language Models with Mixture-of-Experts. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*. PMLR, 5547–5569. https://proceedings.mlr.press/v162/du22c.html

[4] Zhiyuan Fang, Yuegui Huang, Zicong Hong, Yufeng Lyu, Wuhui Chen, Yue Yu, Fan Yu, and Zibin Zheng. 2025. Klotski: Efficient Mixture-of-Expert Inference via Expert-Aware Multi-Batch Pipeline. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS 2025)*. Association for Computing Machinery, 574–588. https://doi.org/10.1145/3676641.3716261

[5] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39. https://www.jmlr.org/papers/v23/21-0998.html

[6] Abhishek Ghosh, Ajay Nayak, Ashish Panwar, and Arkaprava Basu. 2026. GraCE: Unlocking CUDA Graphs with Compiler Support for ML Workloads. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, 1927–1947. https://www.usenix.org/conference/osdi26/presentation/ghosh

[7] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-gated MoE: An Algorithm-System Co-Design for Fast and Scalable Mixture-of-Expert Inference. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1018–1031. https://doi.org/10.1109/ISCA59077.2024.00078

[8] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixtral of Experts. *arXiv preprint arXiv:2401.04088* (2024). https://doi.org/10.48550/arXiv.2401.04088

[9] Rui Kong, Yuanchun Li, Qingtian Feng, Weijun Wang, Xiaozhou Ye, Ye Ouyang, Linghe Kong, and Yunxin Liu. 2024. SwapMoE: Serving Off-the-shelf MoE-based Large Language Models with Tunable Memory Budget. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 6710–6720. https://doi.org/10.18653/v1/2024.acl-long.363

[10] Han Li, Jingwei Sun, Junqing Lin, and Guangzhong Sun. 2026. CommitMoE: Efficient Fallback-Free MoE Inference with Offloading Under GPU Memory Constraints. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 22904–22912. Issue 27. https://doi.org/10.1609/aaai.v40i27.39454

[11] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. 2023. Accelerating Distributed MoE Training and Inference with Lina. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, 945–959. https://www.usenix.org/conference/atc23/presentation/li-jiamin

[12] Guangda Liu, Wenhao Chen, Chengwei Li, Zhenyu Ning, Jing Lin, Yiwu Yao, Quan Chen, Shixuan Sun, Jieru Zhao, and Minyi Guo. 2026. ECHO: Efficient KV Cache Offloading with Lossless Prefetching for Serving Native Sparse Attention LLMs. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, 17–37. https://www.usenix.org/conference/osdi26/presentation/liu-guangda

[13] NVIDIA Corporation. 2026. CUDA C++ Best Practices Guide. NVIDIA CUDA Documentation. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/ Accessed: 2026-08-18.

[14] NVIDIA Corporation. 2026. PyTorch CUDA Graph Integration. CUDA Graph Best Practice for PyTorch. https://docs.nvidia.com/dl-cuda-graph/latest/torch-cuda-graph/torch-integration.html Accessed: 2026-08-18.

[15] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*. PMLR, 18332–18346. https://proceedings.mlr.press/v162/rajbhandari22a.html

[16] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=B1ckMDqlg

[17] Haichen Shen, Jared Roesch, Zhi Chen, Wei Chen, Yong Wu, Mu Li, Vin Sharma, Zachary Tatlock, and Yida Wang. 2021. Nimble: Efficiently Compiling Dynamic Neural Networks for Model Inference. In *Proceedings of Machine Learning and Systems*, Vol. 3. 208–222. https://proceedings.mlsys.org/paper_files/paper/2021/file/5b47430e24a5a1f9fe21f0e8eb814131-Paper.pdf

[18] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 31094–31116. https://proceedings.mlr.press/v202/sheng23a.html

[19] Xiaoniu Song, Zihang Zhong, Rong Chen, and Haibo Chen. 2024. ProMoE: Fast MoE-Based LLM Serving Using Proactive Caching. *arXiv preprint arXiv:2410.22134* (2024). https://doi.org/10.48550/arXiv.2410.22134

[20] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. 2024. HOBBIT: A Mixed Precision Expert Offloading System for Fast MoE Inference. *arXiv preprint arXiv:2411.01433* (2024). https://doi.org/10.48550/arXiv.2411.01433

[21] vLLM Project. 2026. Prefetch-based CPU Offloading in vLLM. https://github.com/vllm-project/vllm/blob/main/vllm/model_executor/offloader/prefetch.py. Accessed: 2026-08-25.

[22] vLLM Project. 2026. UVA Offloader. vLLM Documentation. https://docs.vllm.ai/en/stable/api/vllm/model_executor/offloader/uva/ Accessed: 2026-08-18.

[23] Wenxin Wang, Yule Hou, Yu Ji, Peng Qu, and Youhui Zhang. 2026. Achieving Cloud-Grade SLOs for Local Mixture-of-Experts Inference through CPU–GPU Hybrid Design. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, 1089–1106. https://www.usenix.org/conference/osdi26/presentation/wang-wenxin

[24] Jiale Xu, Rui Zhang, Cong Guo, Weiming Hu, Zihan Liu, Feiyang Wu, Yu Feng, Shixuan Sun, Changxu Shao, Yuhong Guo, Junping Zhao, Ke Zhang, Minyi Guo, and Jingwen Leng. 2024. vTensor: Flexible Virtual Tensor Management for Efficient LLM Serving. *arXiv preprint arXiv:2407.15309* (2024). https://doi.org/10.48550/arXiv.2407.15309

[25] Leyang Xue, Yao Fu, Zhan Lu, Chuanhao Sun, Luo Mai, and Mahesh K. Marina. 2024. MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache. *arXiv preprint arXiv:2401.14361* (2024). https://doi.org/10.48550/arXiv.2401.14361

[26] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie W. Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. In *Proceedings of Machine Learning and Systems*, Vol. 7. MLSys. https://proceedings.mlsys.org/paper_files/paper/2025/file/dbf02b21d77409a2db30e56866a8ab3a-Paper-Conference.pdf